Topic 5: Sinusoidal Model

From spectral peaks to time-varying sinusoidal tracks and additive resynthesis

Digital Signal Processing for Sound and Music · Topic 5

Xavier Serra · Universitat Pompeu Fabra

Lecture roadmap

1 **Model idea**

time-varying sinusoids

2 **Spectral peaks**

windows, lobes, resolution

3 **Peak detection**

zero-padding + interpolation

4 **Tracks**

connect peaks across frames

5 **Synthesis**

main-lobe generation + OLA

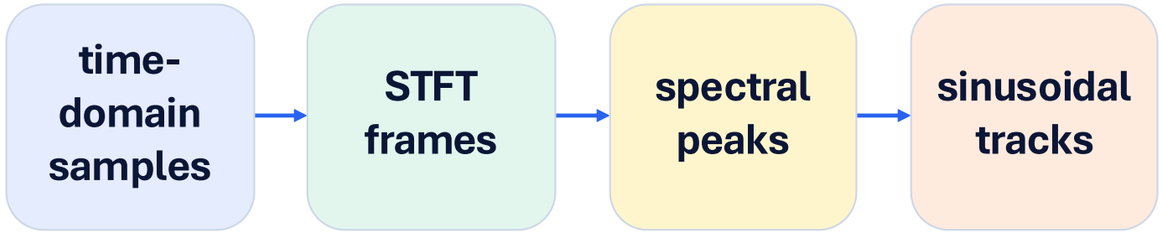
6 **Examples + Lab 5**

inspect, listen, tune parameters

From STFT to a higher-level representation

The sinusoidal model keeps the prominent, interpretable spectral structure

- STFT keeps a full complex spectrum for every frame.
- Sinusoidal modeling keeps selected peaks and tracks.
- We lose some identity information, but gain abstraction and flexibility.



What changes in the representation?

- from one spectrum column to a list of peaks
- from frame-level values to continuous tracks
- from analysis plots to synthesis parameters

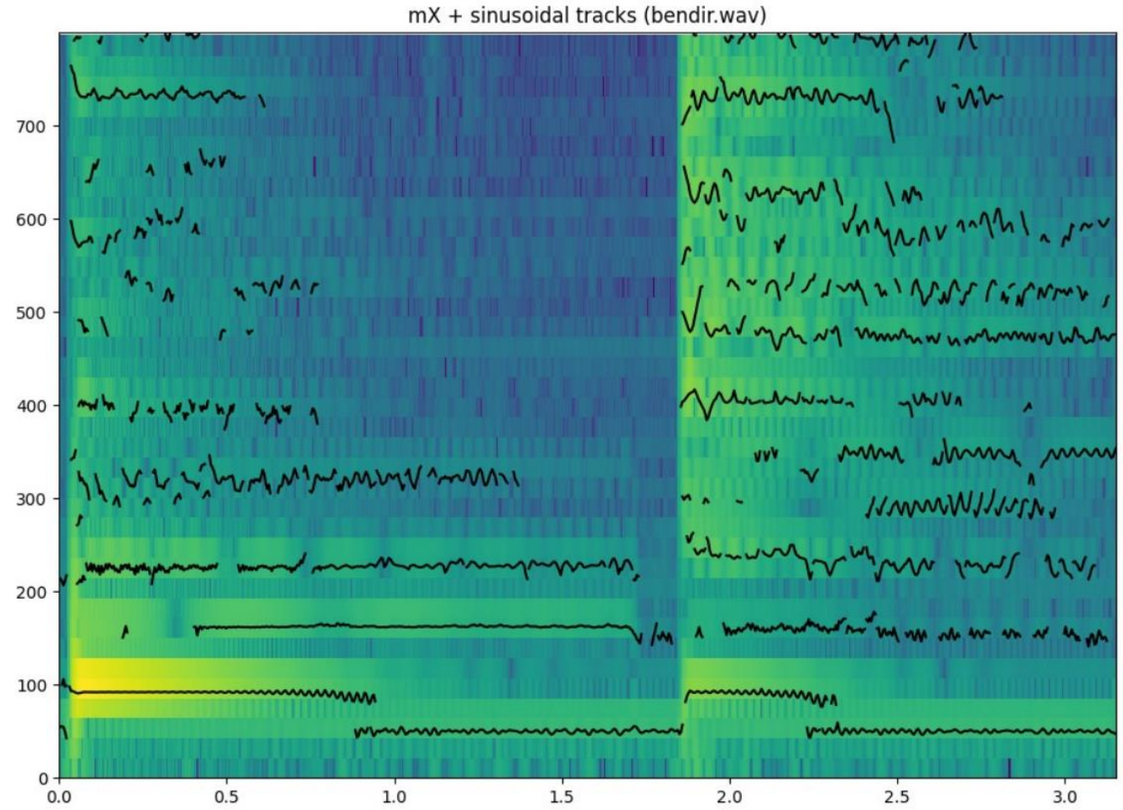
Core idea: find peaks, link peaks, synthesize tracks.

Sinusoidal model

A sound is represented as a sum of time-varying sinusoids

$$y[n] = \Sigma_r \, A_r[n] \cos(2\pi \, f_r[n] \, n + \varphi_r[n])$$

- r indexes the sinusoidal component
- $A_r[n]$ is instantaneous amplitude
- $f_r[n]$ is instantaneous frequency
- $\phi_r[n]$ is phase



Tracks show frequency over time; magnitude and phase are also stored.

Computing the spectrum of a sinewave

Windowed cosine → Euler decomposition → two shifted window spectra

$$x_w[n] = A\,w[n]\cos\left(\frac{2\pi k_0 n}{N} + \phi\right)$$

$$= \frac{A}{2}\,w[n]\left(e^{j(2\pi k_0 n/N+\phi)} + e^{-j(2\pi k_0 n/N+\phi)}\right)$$

$$X[k] = \sum_{n=0}^{N-1} x_w\,[n]\,e^{-j2\pi kn/N}$$

$$= \frac{A}{2}e^{j\phi}\sum_{n=0}^{N-1} w\,[n]\,e^{-j2\pi(k-k_0)n/N}$$

$$+\frac{A}{2}e^{-j\phi}\sum_{n=0}^{N-1} w\,[n]\,e^{-j2\pi(k+k_0)n/N}$$

$$= \frac{A}{2}e^{j\phi}W[k-k_0] + \frac{A}{2}e^{-j\phi}W[k+k_0]$$

Key idea

- start from a finite cosine
- split it with Euler's formula
- collect the two complex exponentials
- each term becomes a shifted copy of W[k]
- result: two symmetric lobes at $\pm k_0$

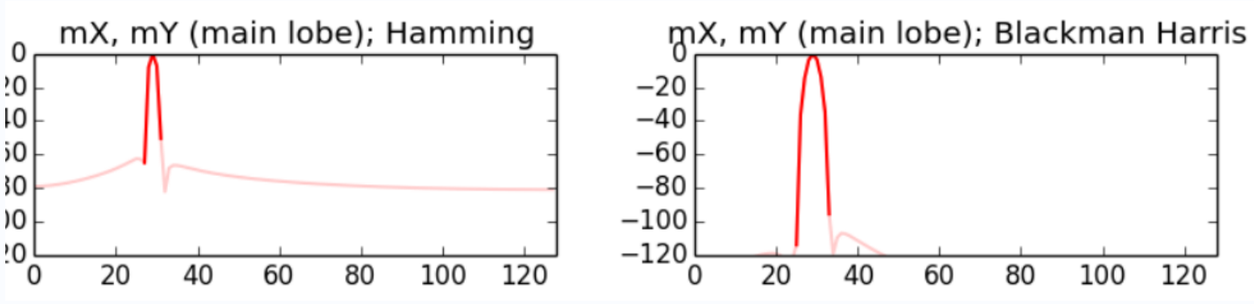
Windowing shapes every sinusoidal peak

The main-lobe width determines the minimum M needed to separate nearby components

Window main lobe and bin spacing

$B_f = B_s \cdot f_s / M$
To resolve: $B_f \le \Delta$
$\Delta = |f_{k+1} - f_k|$
$M \ge B_s \cdot f_s / \Delta$

B_s: main-lobe width in bins
f_s: sampling rate
M: window size
Δ: frequency separation



Different windows create different main-lobe widths and side-lobe levels.

Harmonic sounds

Successive harmonics: $\Delta = f_1$

Period in samples: $P = f_s / f_1$

$$M \ge B_s \cdot f_s / f_1 = B_s \cdot P$$

Hamming: $B_s \approx 4 \rightarrow M \ge 4$ periods

Example $f_1 = 110$ Hz, $f_s = 44{,}100$ Hz

$P \approx 401$ samples → $M \ge 1604$

Resolution guideline; changes in time also matter.

Choosing M is a modeling decision

A wrong window size can merge, smear, or distort sinusoidal components

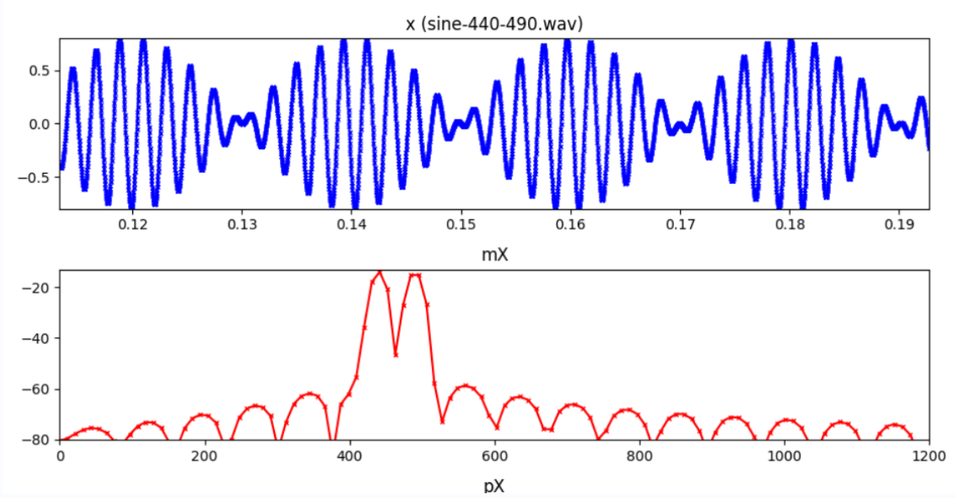
Example: 440 Hz and 490 Hz

Hamming window: $B_s = 4$
$f_s = 44100$ Hz
$\Delta = |490 - 440| = 50$ Hz

$$M \geq B_s \cdot f_s / \Delta$$
$$M \geq 4 \cdot 44100 / 50$$
$$M \geq 3528 \text{ samples}$$

M ≥ 3528 samples (≈ 80 ms): frequency precision costs time precision.

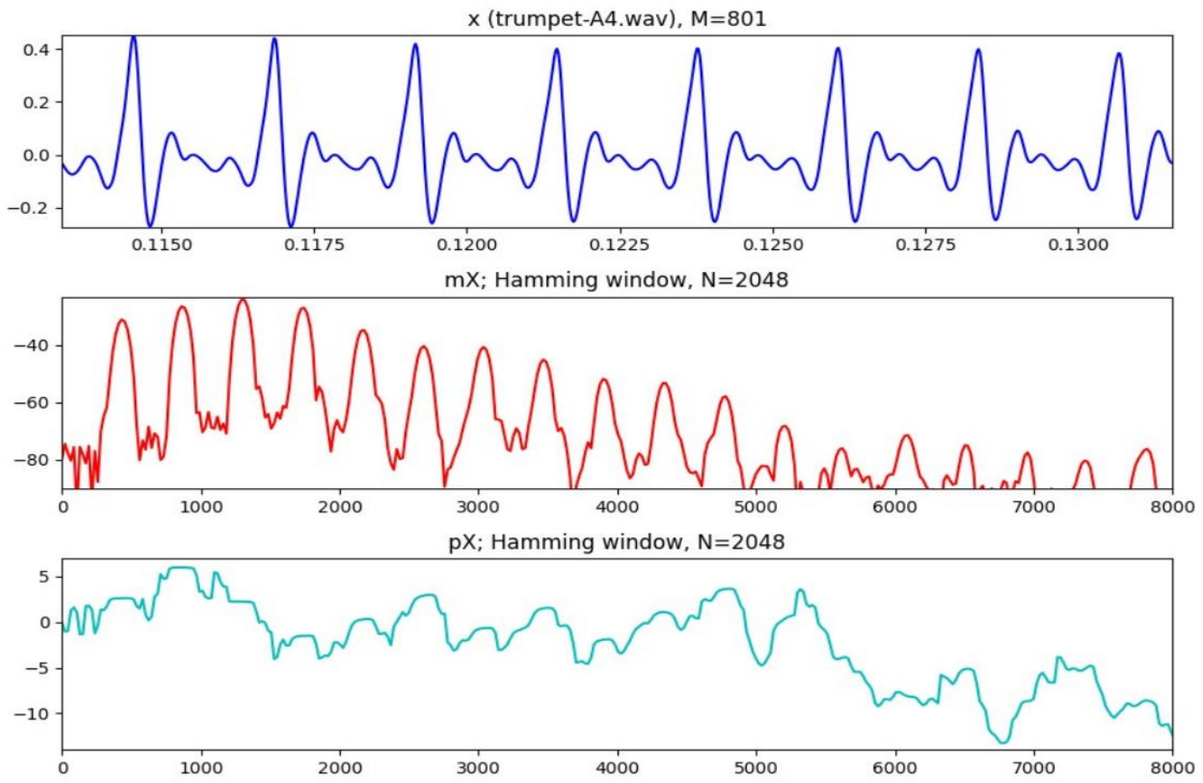
Plot check: two resolvable peaks



The waveform shows beating; the magnitude spectrum should show two separate peaks.

Real musical sounds contain many overlapping peaks

The sinusoidal model is useful when the important structure can be tracked as peaks



trumpet: spectral peaks and phase

Sinusoids versus physical partials

A spectral peak is a measurement; a sinusoidal track is a model; a partial is a source component

SINUSOID · representation

A modeled component with frequency, magnitude and phase evolving in time.

- Mathematical building block of additive synthesis.
- A spectral peak may reflect leakage, noise, or a physical partial.
- Adjacent DFT bins of one window lobe do not represent distinct sinusoids.

PARTIAL · physical interpretation

A sound-source component, often linked to a vibrational mode.

- Partials may be harmonic or inharmonic (e.g., bells).
- Identify them from spectral shape, persistence and relationships to other components.
- Sound-source knowledge can support interpretation.

A stable physical partial can often be approximated by one sinusoidal track — but not every detected peak is a partial.

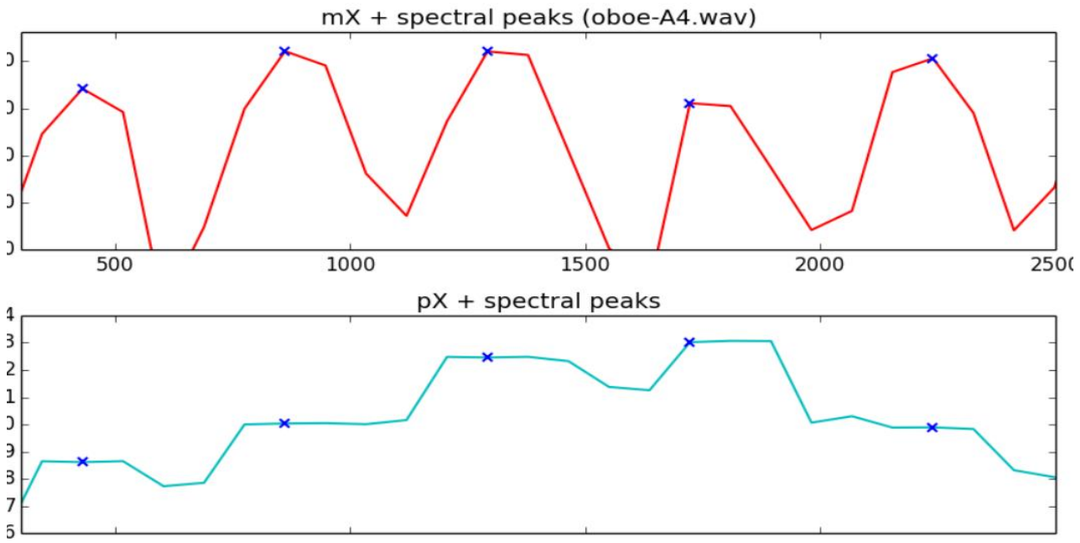
Peak detection

Detect local maxima above a threshold before refining their frequency estimates

peak at bin k if

$$mX[k] > t$$

$$mX[k] > mX[k-1] \quad \text{and} \quad mX[k] > mX[k+1]$$

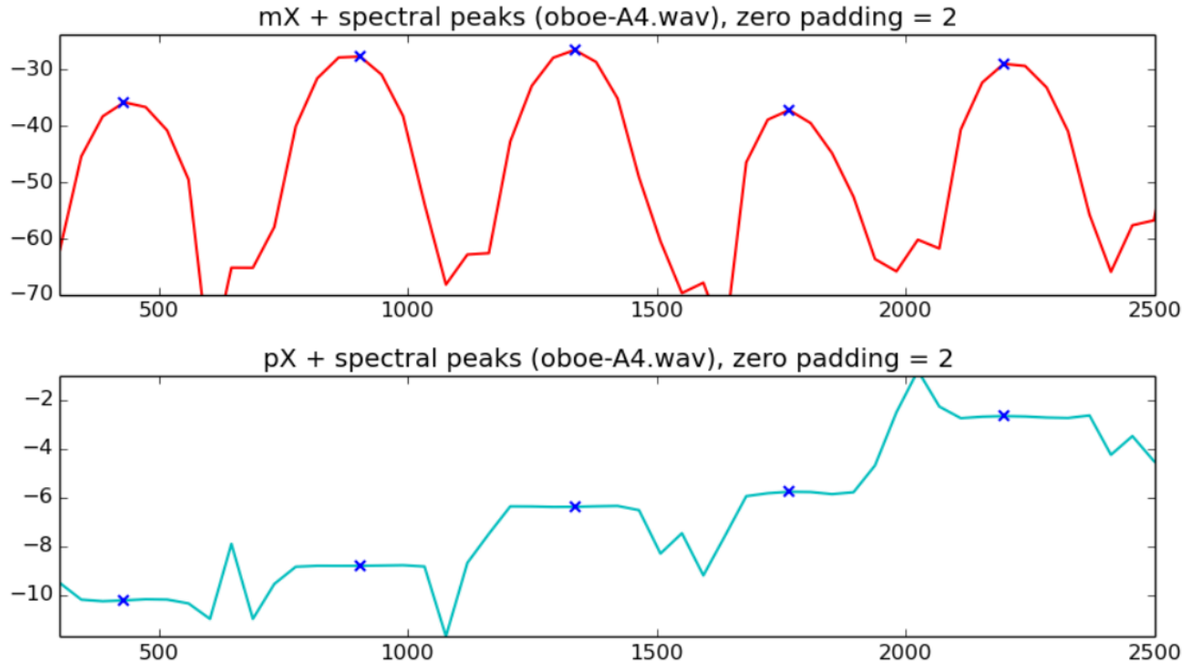


Interpretation

- mX[k] and threshold t are in dB.
- k is a DFT-bin index, not an exact frequency.
- For an ideal isolated peak, bin-grid quantization is at most ½ bin (not ½ time sample).
- Zero-padding gives a denser grid, but does not improve intrinsic resolution.
- peakInterp() refines magnitude, bin location, and phase.

Resolution problem: zero-padding helps but is not enough

Zero-padding samples the DFT more densely, but it can become computationally expensive



same peak with a denser FFT grid

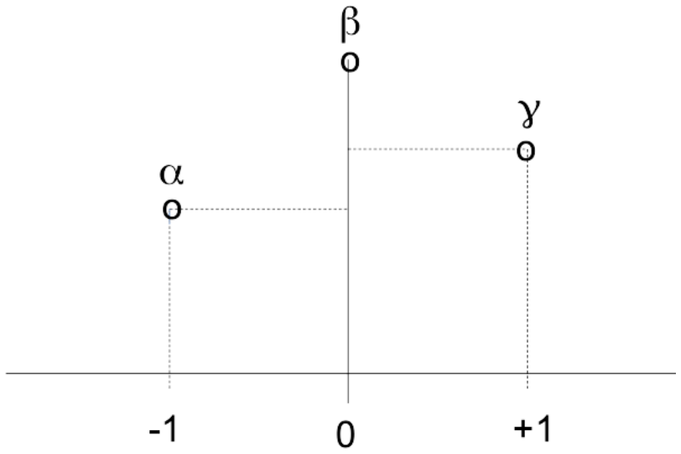
Correct interpretation

- N controls the FFT sampling grid
- M still controls the windowed frame
- more bins do not add new information
- interpolation refines the peak estimate cheaply

Parabolic interpolation refines the peak tip

Use the highest bin and its two neighbors to estimate fractional bin location and amplitude

Three-point interpolation



$$\alpha = mX[k_p - 1] \quad \beta = mX[k_p] \quad \gamma = mX[k_p + 1]$$

These three values define the local parabola around the detected maximum.

Parabolic model and outputs

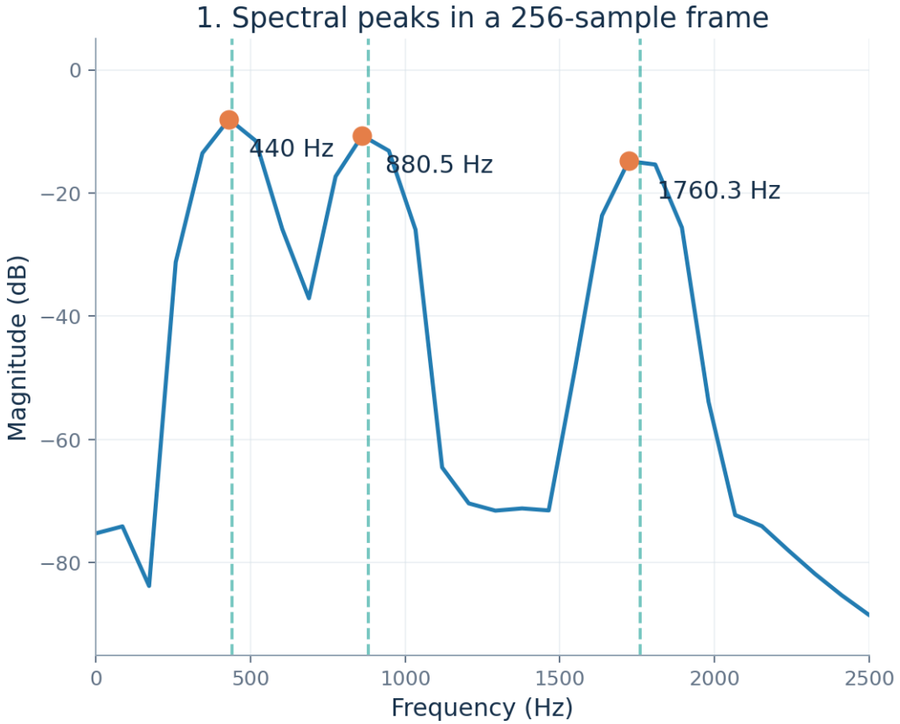
$$p(n) = a(n - \hat{p})^2 + \hat{b}$$

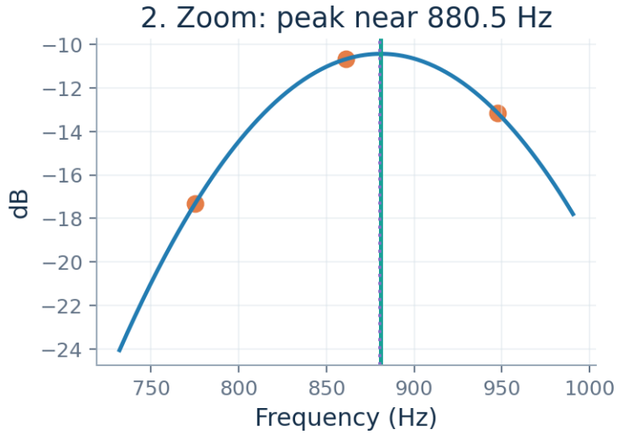
- $\hat{p}$ is the fractional offset from the central bin
- $\hat{b}$ is the refined peak magnitude
- the refined frequency uses $\hat{k} = k_p + \hat{p}$
- phase is then evaluated at the refined position

$$\hat{p} = 1/2 \cdot (\alpha - \gamma)/(\alpha - 2\beta + \gamma)$$
$$\hat{b} = \beta - 1/4 \cdot (\alpha - \gamma)\hat{p}$$

Notebook example · Why interpolate peaks?

Original tones: 440, 880.5 and 1760.3 Hz · Blackman window · M = N = 256 · fs = 22050 Hz





Original	FFT bin	Interpolated
440.0 Hz	430.7 Hz	440.3 Hz
880.5 Hz	861.3 Hz	881.0 Hz
1760.3 Hz	1722.7 Hz	1760.6 Hz

Notice: an FFT-bin maximum is a discrete estimate; a 3-point parabola moves the estimate between bins.

Source: examples/peak-detection-process.ipynb — recreated from the notebook's signal and parameters

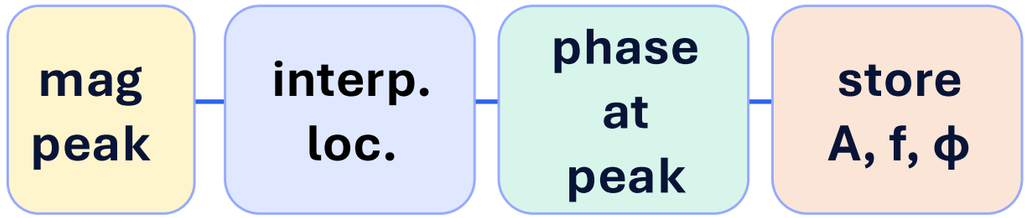
From one frame to sinusoidal parameters

Each accepted peak becomes a local estimate of frequency, amplitude, and phase

Equations used in the model

$$\hat{k}_p = k_p + \hat{p}$$
$$f_p = f_s \cdot \hat{k}_p / N$$
$$A_p = \hat{m}_p$$
$$\varphi_p = \angle X[\hat{k}_p]$$

f_p: estimated frequency
A_p: estimated amplitude
ϕ_p: phase read at the refined peak

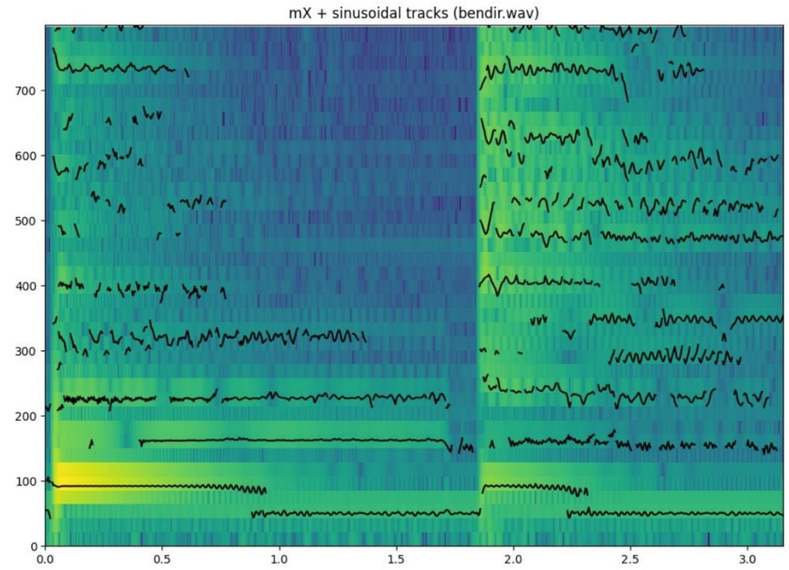


- A single frame gives isolated estimates.
- A model needs these estimates connected over time.

From peaks to sinusoidal tracks

In sms-tools, peaks are assigned by frequency proximity, starting with the strongest

frame ℓ peaks → **sort by magnitude** → **nearest active track** → **frequency tolerance** → **carry f, A, ϕ forward**



Track assignment uses frequency only; magnitudes prioritize peaks and phases are passed through.

sineTracking(): greedy peak-to-track matching

The sms-tools implementation links the current frame to the previous frame

Inputs and outputs

current peaks: pfreq, pmag, pphase | previous tracks: tfreq

1 ORDER Process detected peaks by descending magnitude.

2 ASSIGN Find the closest unused, previously active track.

3 CHECK Keep that identity only if the frequency difference is small enough.

4 UPDATE Transfer its f, magnitude and phase; unmatched peaks start tracks.

No phase or amplitude criterion in track matching; they are copied to the output.

Frequency gate

$$|f_p - f_{track}| < d(f_p)$$

$d(f_p) = \text{offset} + \text{slope} \cdot f_p$

Defaults: 20 Hz + 0.01 · f_p

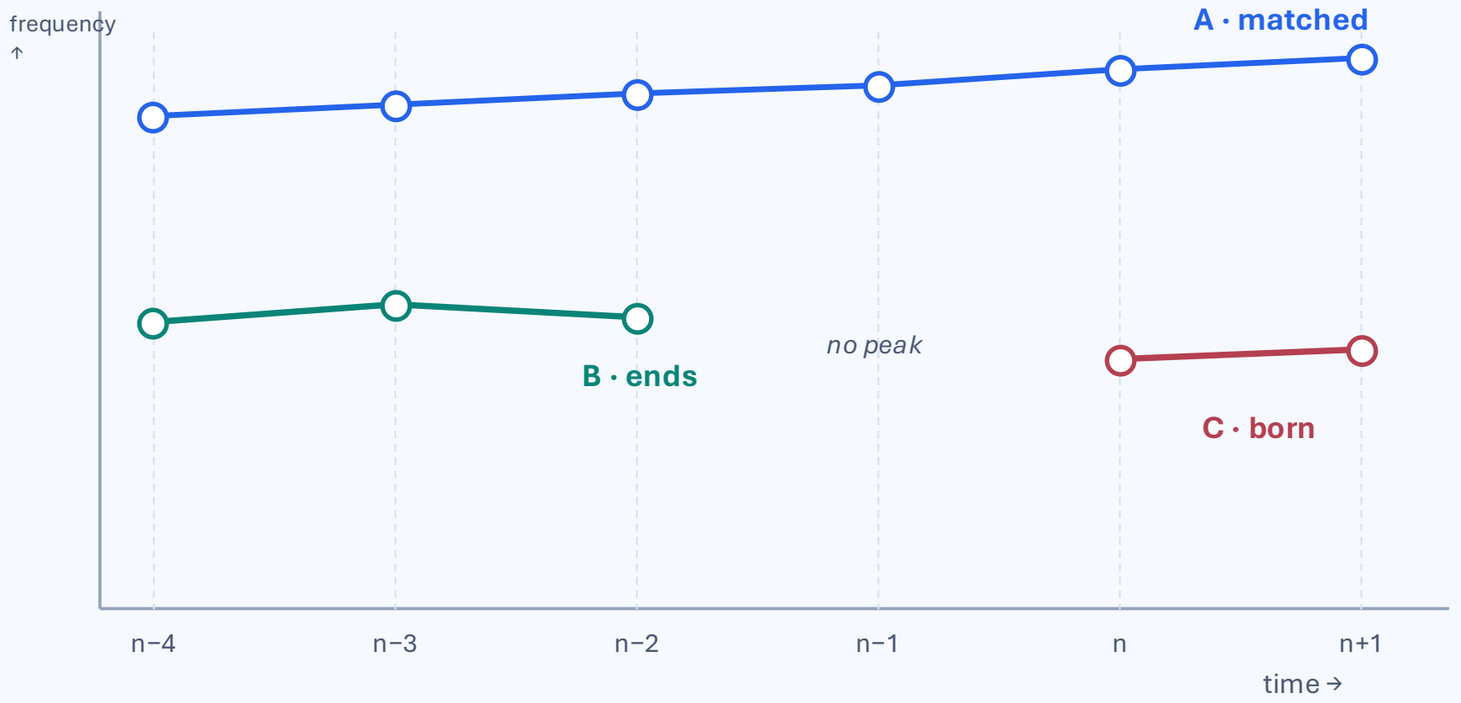
At 1000 Hz: tolerance = 30 Hz

1024 Hz: match 1050 Hz: no match

Track identities, births, and endings

A new frame either continues a track, starts one, or leaves a previous track inactive

Frequency trajectories across analysis frames



B has no peak at n−1; a peak at n starts C, not a revived B.

What the code does

MATCH

Nearest unused active track passes the frequency gate.

END

Unmatched previous track becomes zero in this frame.

START

Remaining peaks use empty/new slots (then capped by maxnSines).

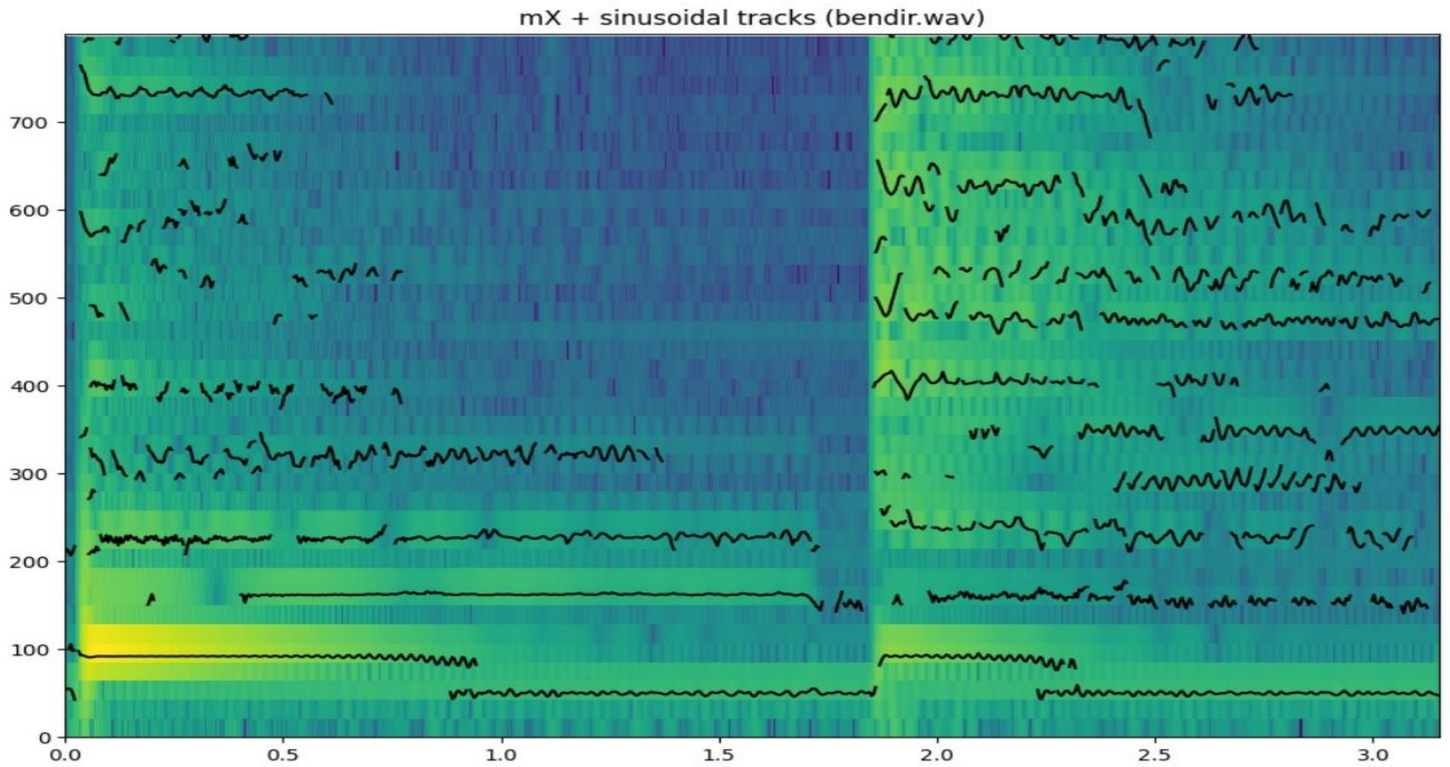
CLEAN

sineModelAnal removes very short track fragments.

No “sleeping guides” in this implementation.

Example: magnitude tracks on a spectrogram

Use the magnitude spectrogram to see where the tracked components follow strong energy ridges



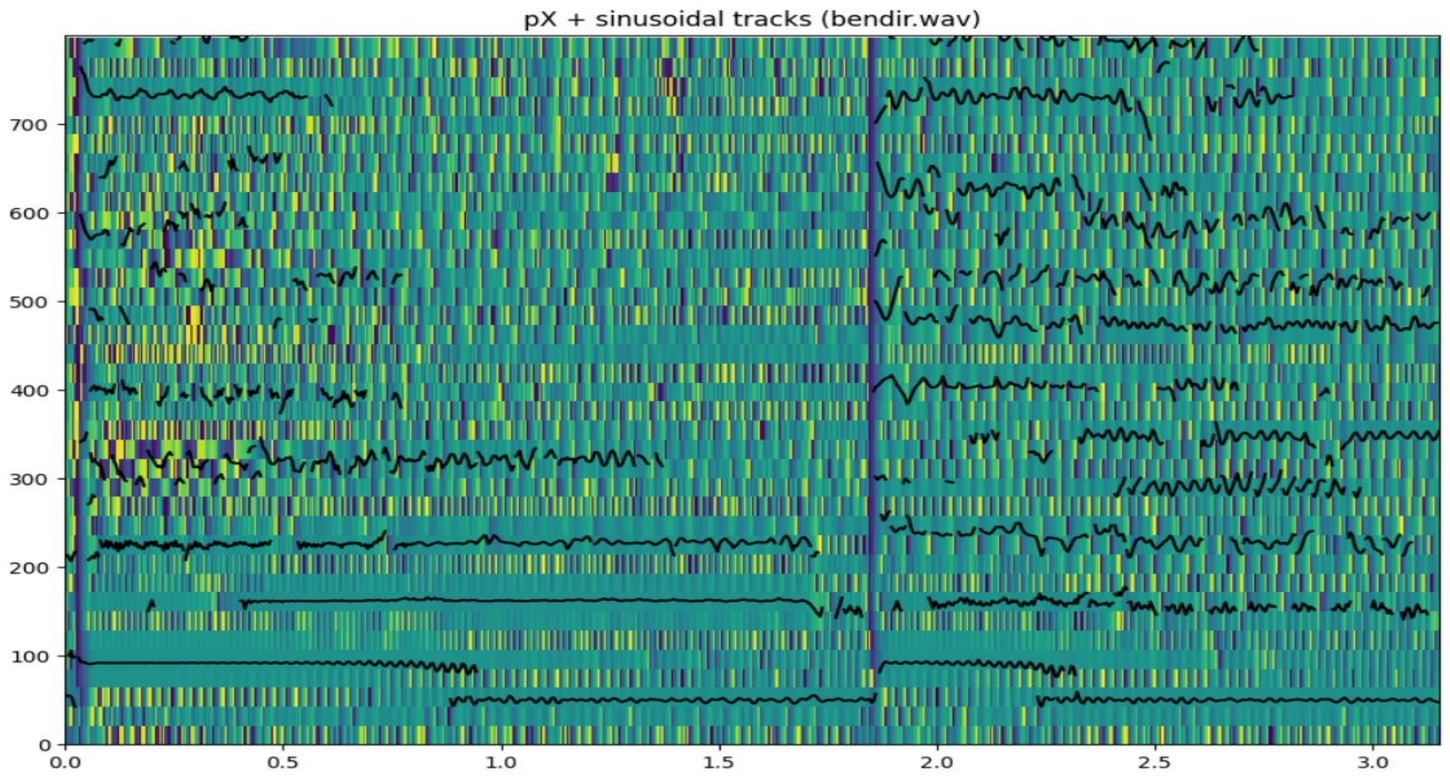
bendir example: magnitude spectrogram + same track positions

How to read it

- background color shows magnitude
- black curves are sinusoidal tracks
- strong ridges should support stable tracks
- gaps often signal weak or noisy regions

Phase is useful for checking sinusoidal consistency

A track is more reliable when magnitude and phase agree over time



bendir example: phase spectrogram + same track positions

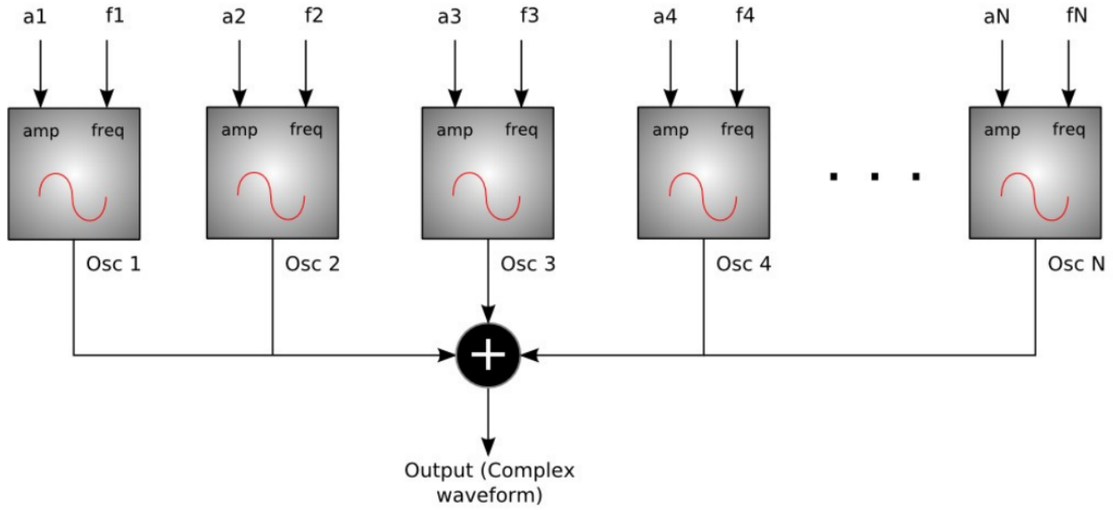
Why not magnitude only?

- phase carries temporal alignment
- abrupt phase behavior may reveal wrong tracks
- phase is meaningful mainly near strong peaks
- use it as a consistency check, not alone

Synthesis: parameters become sound again

After analysis, the model must generate a waveform from tracks

Sinusoidal (additive) synthesis



http://en.wikibooks.org/wiki/File:SSadditiveblock.png

Synthesis question

- for each time frame, generate the sinusoidal components
- combine them into a time-domain frame
- overlap-add frames into a continuous signal

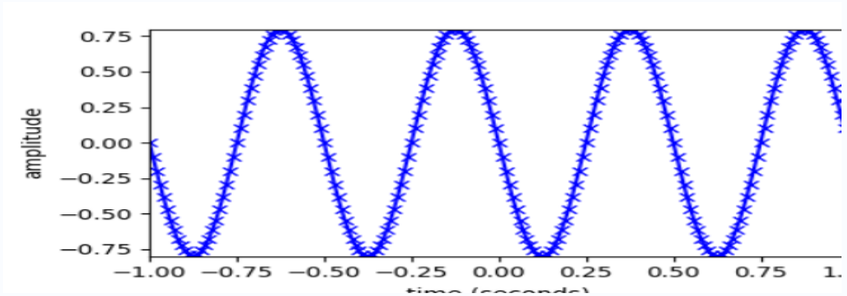
Synthesizing one sinusoid

Direct time-domain generation is simple, but spectral synthesis connects to the model

Time-domain generation

$$y[n] = A_r[n] \cos(2\pi f_r[n] \, n / f_s + \varphi_r)$$

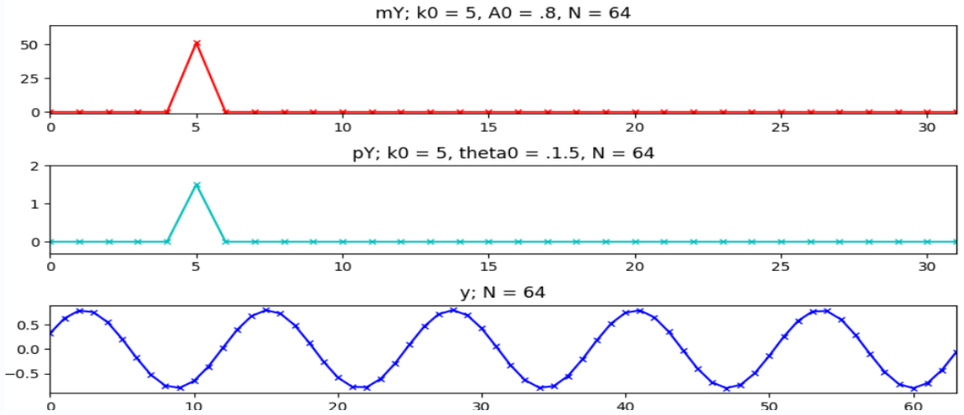
$A_r[n]$: instantaneous amplitude
$f_r[n]$: instantaneous frequency
ϕ_r: initial phase



A sinusoid can be generated directly from amplitude, frequency, and phase.

On-bin spectral synthesis

$$y[n] = \mathrm{IDFT}\{\, mY[k] \cdot e^{jpY}[k] \,\}$$
$$mY[k_0] = A_0, \quad pY[k_0] = \varphi_0$$
$$mY[k] = 0 \text{ for } k \neq k_0$$



One spectral bin gives one bin-centered sinusoid after the inverse DFT.

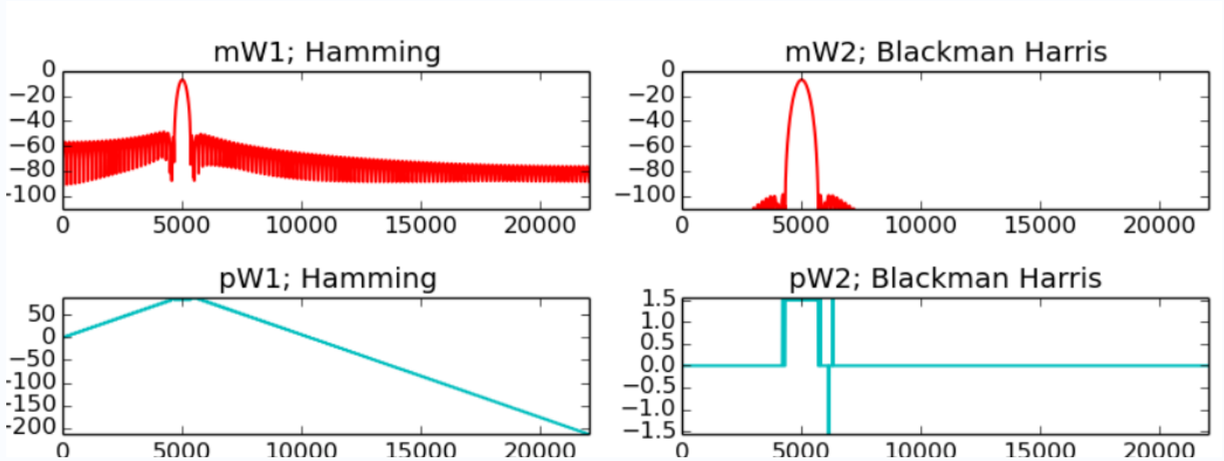
Any frequency requires a shaped spectral lobe

Off-bin sinusoids are not represented by one isolated DFT sample

Spectral synthesis at an arbitrary frequency

$$Y[k] = A_0 \cdot |W[k - k_0]| \cdot e^{j(\angle W[k-k_0] + \varphi_0)}$$
$$y[n] = \mathrm{IDFT}\{\, Y[k] \,\}$$

$mW[k]$, $pW[k]$: magnitude and phase of the window spectrum



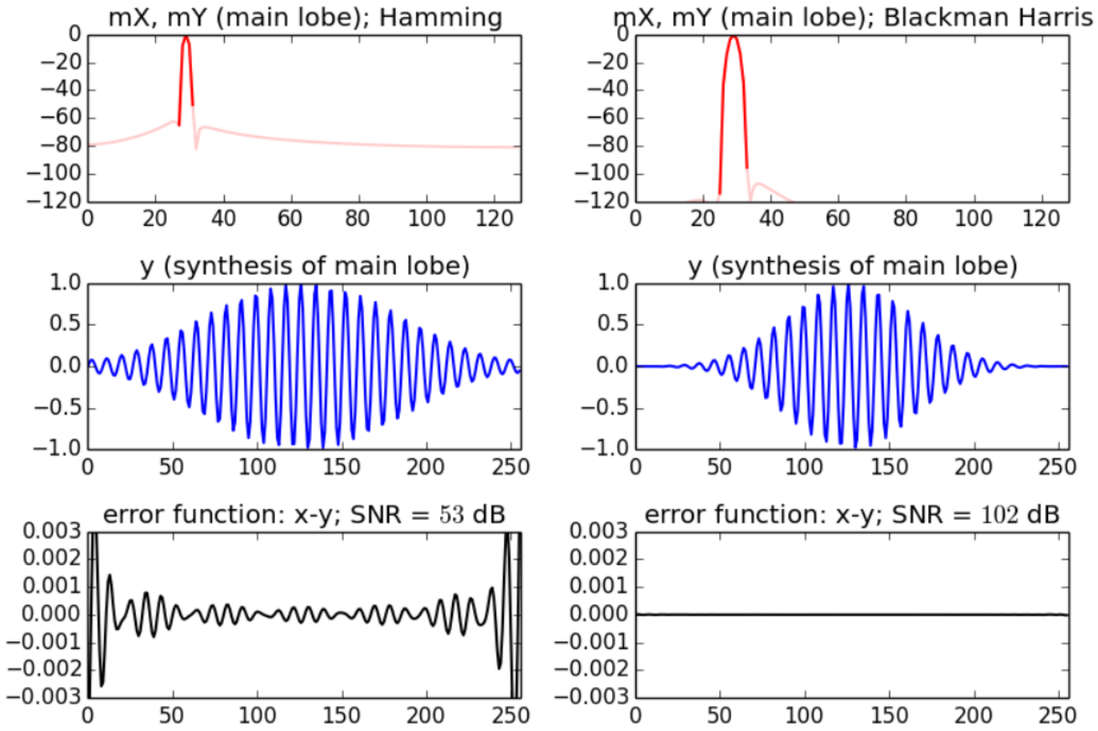
The shifted window spectrum defines magnitude and phase around the target frequency.

Synthesis rule

1. compute the window transform
2. shift it to the target frequency
3. set amplitude and phase
4. apply the inverse DFT

Main-lobe synthesis

In practice most energy is concentrated in a few bins around the peak



compare full window spectrum and main-lobe-only synthesis

Practical choice

- Hamming: first side-lobe around −42 dB
- Blackman-Harris: much lower side-lobes
- wider main lobe costs more bins
- good synthesis often uses only the main lobe

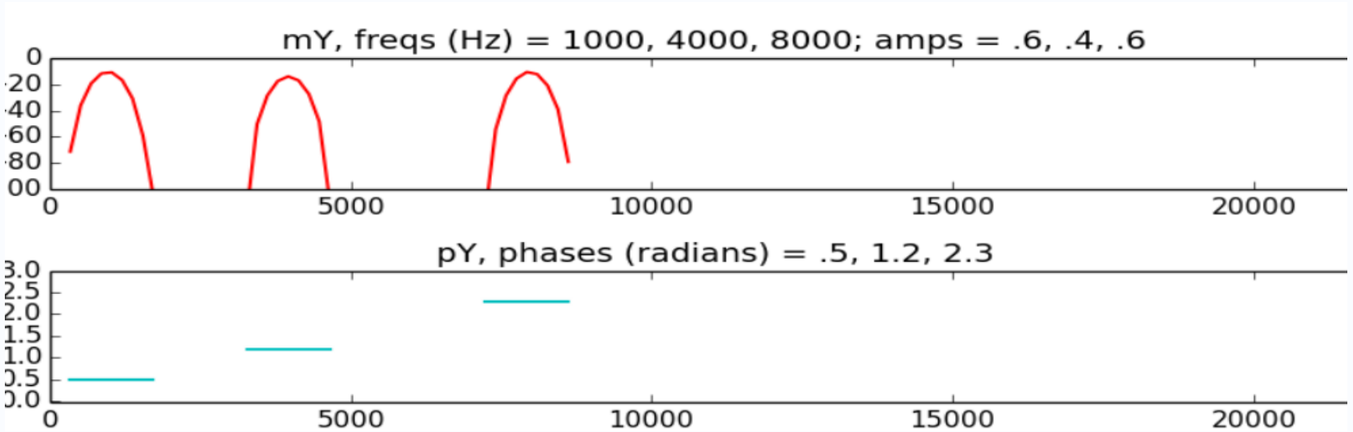
Additive synthesis in the spectrum

Several sinusoids are generated by adding several shifted main lobes

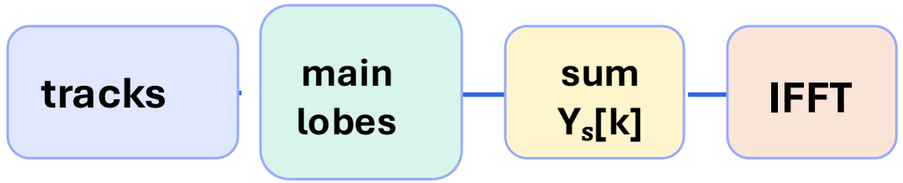
$$Y_+[k] = \sum_{r=1}^{R} \frac{A_r}{2} e^{j\phi_r} W[k - k_r]$$

$$Y_s[k] = Y_+[k] + Y_+^*[(-k) \bmod N]$$

$$y[n] = \mathrm{IDFT}\{Y_s[k]\}$$



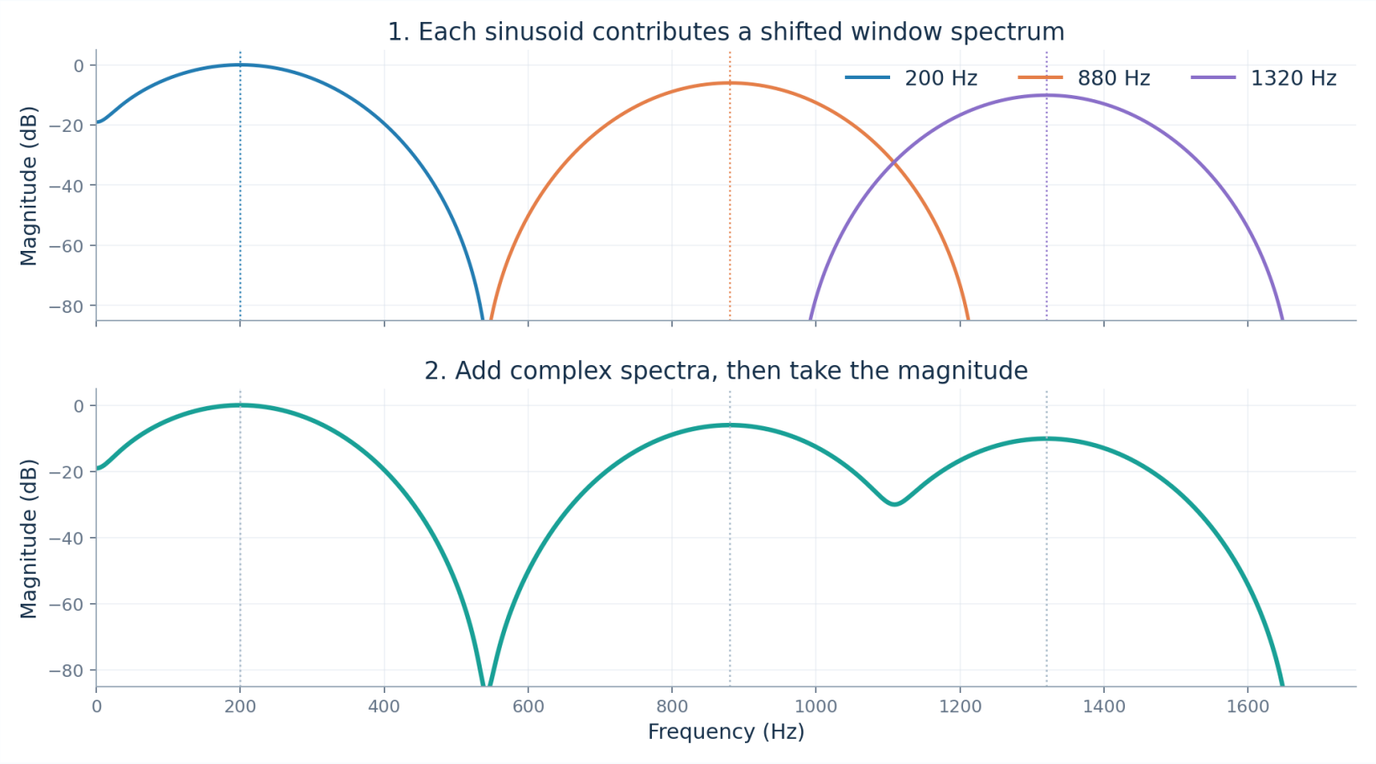
Each sinusoid contributes one shifted lobe with its own amplitude and phase.



Build each lobe, add its mirror, then apply the IFFT

Notebook example · Three spectral lobes become one spectrum

Three sinusoids: 200, 880 and 1320 Hz · amplitudes 0.8, 0.4, 0.25 · FFT synthesis size 512



Add COMPLEX contributions before taking the magnitude: dB magnitudes cannot simply be added.

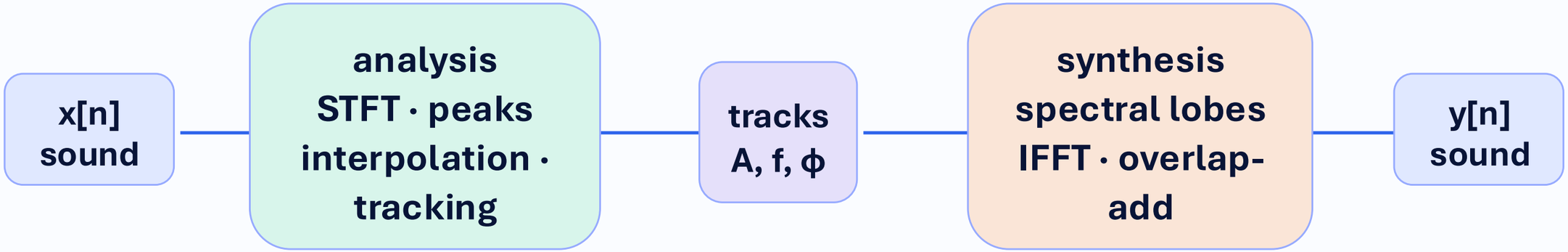
Source: examples/sinusoidal-synthesis-examples.ipynb — illustration recreated from its parameters

Sinusoidal model system

Analysis detects and tracks peaks; synthesis turns tracks back into sound

Analysis side: find meaningful sinusoidal components in each frame and connect them over time.

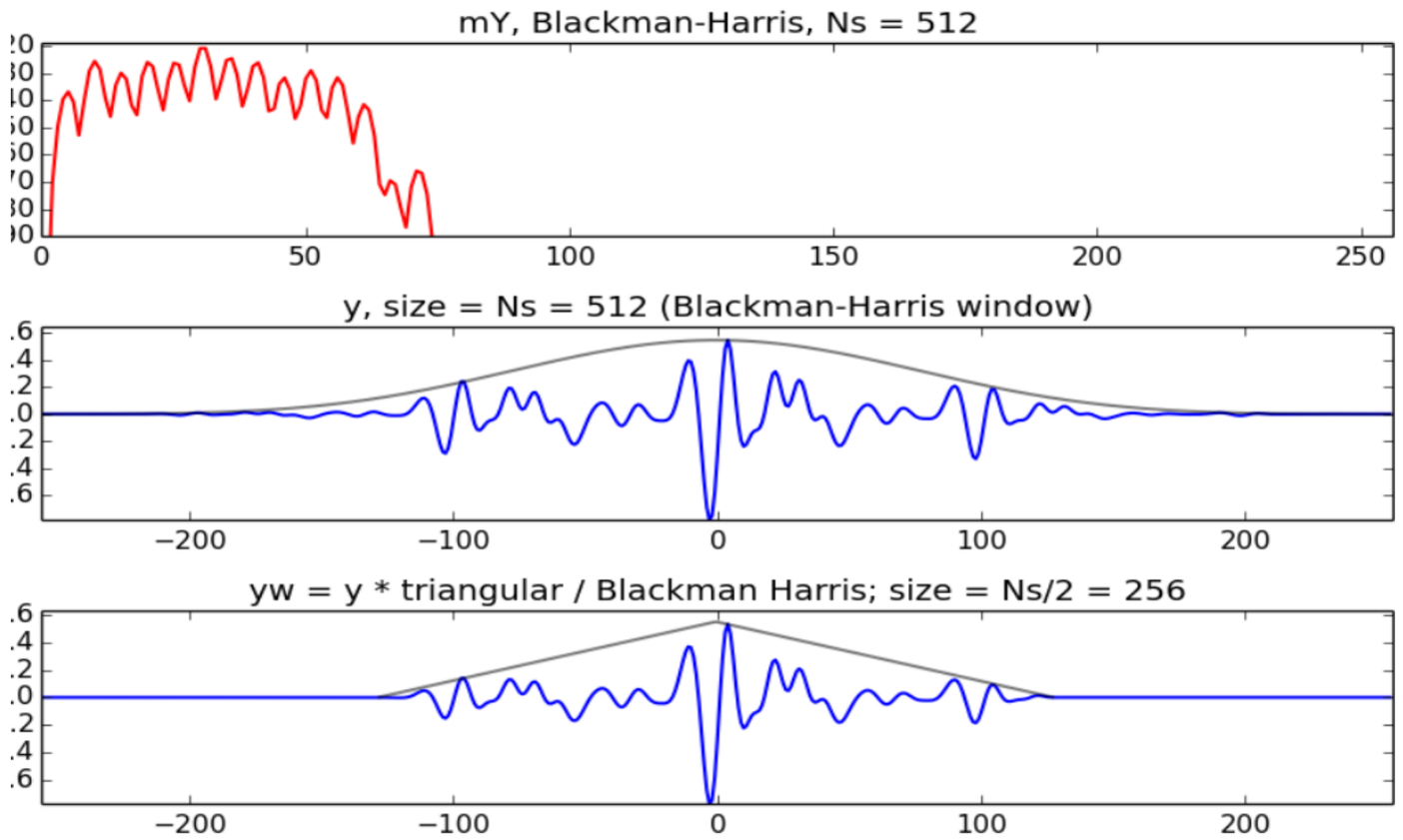
Synthesis side: turn track parameters into spectral components and reconstruct audio frames.



The tracks are the compact model: frequency, magnitude, phase, and identity through time.

Synthesis window and overlap-add

A synthesis window is needed for smooth reconstruction from overlapping frames



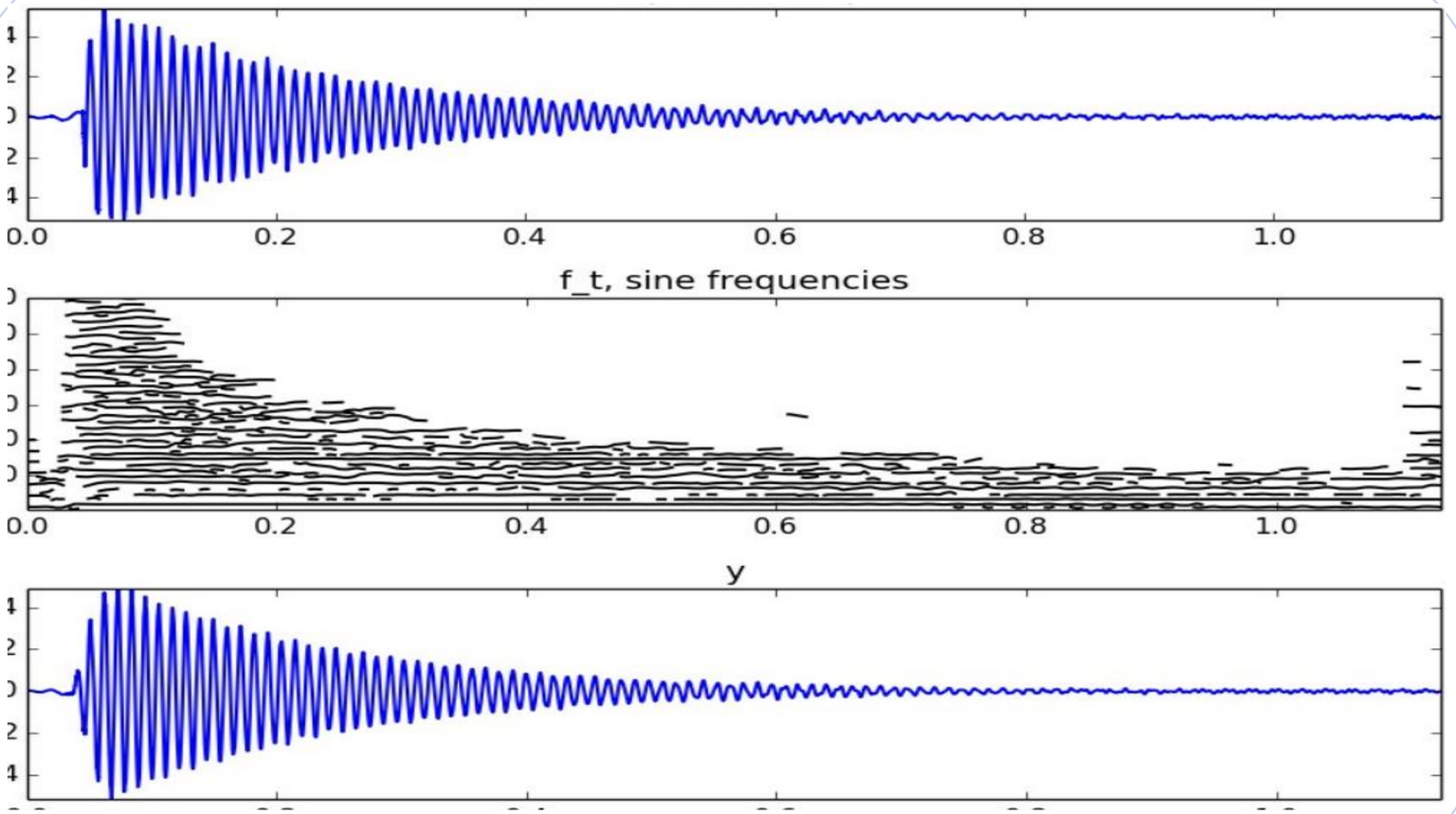
undo analysis window and apply synthesis window

25% overlap example

- generate one frame in the spectrum
- IFFT gives a time-domain segment
- apply the synthesis window
- overlap and add consecutive frames
- the hop H determines the overlap

Example: analysis/synthesis result

The final test is visual and audible: compare tracks, waveform behavior, and listening quality



input sound, sinusoidal tracks, resynthesized output

What to check

- are important tracks continuous?
- are attacks and decays preserved?
- does the output sound close enough?
- which components are missing or unstable?

Example notebooks: where each plot comes from

Use the notebooks to connect equations, arrays, parameters, tracks, and listening

examples/

peak-detection-process.ipynb

Peak detection and interpolation

threshold → local maxima → refined frequency / magnitude / phase

examples/

sine-model-examples.ipynb

Full sinusoidal-model workflow

load sound → analyze → track → synthesize → listen

examples/

sinusoidal-synthesis-examples.ipynb

Synthesis from sinusoidal parameters

A, f, ϕ tracks → spectral lobes → IFFT frames

Implementation · sineModelAnal()

From each frame’s spectral peaks to time-varying sinusoidal tracks

1 Frame FFT
dftAnal()

→

2 Detect peaks
peakDetection()

→

3 Interpolate
peakInterp()

→

4 Link peaks
sineTracking()

→

5 Clean tracks
cleaningSineTracks()

Function call

```
xtfreq, xtmag, xtphase = sineModelAnal(x, fs, w, N, H, t, maxnSines, minSineDur, freqDevOffset, freqDevSlope)
```

Key parameters

w / N / H / t: window, FFT size, hop, peak threshold
maxnSines: track limit · minSineDur: short-track removal
freqDevOffset / Slope: frequency matching tolerance

Three output matrices

xtfreq [Hz] · xtmag [dB] · xtphase [rad]
Shape: frames × maxnSines (columns = track slots)
A zero frequency marks an inactive track.

Implementation source: github.com/MTG/sms-tools · smstools/models/sineModel.py

Implementation · sineModelSynth()

Convert tracked frequency, magnitude, and phase into a reconstructed waveform

Function call

```
y = sineModelSynth(tfreq, tmag, tphase, N, H, fs)
```

For each synthesis frame

1 Track parameters

f · magnitude · phase

→

2 genSpecSines()

sum complex spectral lobes

→

3 IFFT + fftshift

time-domain frame

→

4 Window + overlap-add

accumulate every H samples

Phase handling

Use supplied phases when available; otherwise advance phases using adjacent-frame frequencies.

Synthesis window

_build_synthesis_window() combines triangular and Blackman–Harris shapes before overlap-add.

Implementation source: github.com/MTG/sms-tools · smstools/models/sineModel.py

Exercise 5: Sinusoidal model

estimate, track, synthesize, and diagnose sinusoids in controlled and musical sounds

What you will practice

- detect and interpolate spectral peaks
- track sinusoids across frames
- tune window, threshold, and tracking parameters
- compare plots with what you hear

How to start

Open exercises/E5-Sinusoidal-model.ipynb

Use: NumPy · Matplotlib · SciPy windows · sineModelAnal() · sineModelSynth()

1 · Estimate

single sinusoid
minimize frequency error

2 · Track

two-component chirp
choose M carefully

3 · Separate

large amplitude range
window + threshold

4 · Use phase

check track stability
avoid magnitude-only decisions

5 · Resynthesize

multiSines analysis
inspect + listen

Lab 5: change parameters deliberately

Each parameter has a visible and audible consequence

parameter	meaning	what to observe
M	window length	frequency separation vs time precision
N	FFT size	spectral sampling and peak interpolation support
t	threshold	reject weak/noisy peaks or keep faint sinusoids
maxnSines	track limit	maximum number of sinusoids per frame
minSineDur	cleaning	remove short spurious tracks
freqDevOffset / Slope	tracking tolerance	allow frequency motion between frames

What to remember from Topic 5

The sinusoidal model is a compact representation built on top of STFT analysis

1 Model
sound as time-varying sinusoids

2 Peaks
frequency, magnitude, and phase from spectra

3 Windows
M and window shape control leakage and resolution

4 Tracks
link frame-level peaks into evolving components

5 Synthesis
main lobes, IFFT, synthesis window, overlap-add

6 Validation
inspect magnitude/phase, tracks, waveform, and listening

References and credits

- sms-tools models: https://github.com/MTG/sms-tools
- sms-tools-materials repository: class slides, example notebooks, and lab materials
  - examples/sine-model-examples.ipynb for analysis, tracking, and inspection
  - examples/peak-detection-process.ipynb for peak detection and interpolation
  - examples/sinusoidal-synthesis-examples.ipynb for spectral sinusoidal synthesis
- Serra, X. and Smith, J.O. (1990). Spectral modeling synthesis: a sound analysis/synthesis system based on a deterministic plus stochastic decomposition.